

Playwright-Notes-4

- **Playwright.config.ts file**- Playwright has many options/settings to configure how your tests run.

-First **import { defineConfig, devices } from '@playwright/test'** include this line at the top of your **Playwright.config.ts** file

-Then export the **defineConfig** component like so:

```
export default defineConfig({  
});
```

- The below options can be specified in the configuration file. Below are some options/configurations/settings:

1. **testDir**- specifies the directory in which test files will be created and executed-tells Playwright where to look for your test files

Note: npx playwright test- runs all your Playwright tests

2. **fullyParallel**- specifies how Playwright runs your tests/testblocks. If set to true, it runs all the tests/test blocks at the same time. If false it runs the tests/test blocks one by one

Eg: // playwright.config.ts

```
import { defineConfig } from '@playwright/test';
```

```
export default defineConfig({  
  fullyParallel:true,  
});
```

3. **forbidOnly**- In Playwright you can add .only to a test to run just that one test and skip everything else like so:

```
test.only('login test', async ({ page }) => {  
  // only this test runs, all others are skipped  
});
```

```
test('dashboard test', async ({ page }) => {
```

```
// this gets skipped  
});
```

This is very handy during development — when you are working on one test and don't want to wait for all the others.

The Problem

Developers often forget to remove `.only` before pushing code to the team.

Now when the whole team runs tests in the CI/CD pipeline (the automated system that runs tests for everyone), only that one test runs — and all other tests are silently skipped. Nobody notices bugs in the skipped tests.

What forbidOnly Does

forbidOnly: true tells Playwright:

"If anyone has left a `.only` in the code, fail immediately and don't run anything."

It is a safety net that stops `.only` from accidentally reaching the team's pipeline.

```
// playwright.config.ts  
import { defineConfig } from '@playwright/test';  
  
export default defineConfig({  
  forbidOnly: !!process.env.CI, //only enforce this on CI/CD  
}); // Local machine → false → .only is allowed  
  // CI/CD pipeline → true → .only causes failure
```

What !!process.env.CI Means

typescriptforbidOnly: !!process.env.CI

process.env.CI — an environment variable that is set to true on CI/CD systems like GitHub Actions, Jenkins etc.

!! — converts it to a proper boolean (true or false)

So in plain English:

"Only forbid `.only` when running in the CI/CD pipeline — allow it on my local machine." `!!process.env.CI` is simply a way of asking "are we running on a CI/CD pipeline right now?" and getting back a clean true or false answer.

4. retries- Tells Playwright how many times to re-run a failed test before marking it as failed.

Eg: // playwright.config.ts
export default defineConfig({
 retries: 2, // try each failed test up to 2 more times
});

Eg: // playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
 retries: process.env.CI ? 2 : 0,
 // On CI/CD → retry twice
 // On local machine → no retries
});

In plain English:

"On my local machine I want to see failures immediately. On the CI/CD pipeline, give tests 2 extra chances in case of flakiness."

Why retries: 0 Locally?

When developing locally you want to see failures straight away so you can fix them immediately — retrying just slows you down.

typescriptretries: process.env.CI ? 2 : 0

// Local → retries: 0 → fail fast, fix fast

// CI/CD → retries: 2 → handle flaky tests gracefully

5. workers- Controls how many tests run at the same time in parallel.

Eg: // playwright.config.ts
export default defineConfig({
 workers: 4, // run 4 tests simultaneously
});

Common Configuration

```
export default defineConfig({
  workers: process.env.CI ? 2 : 4,
  // On CI/CD pipeline → 2 workers
  // On local machine → 4 workers
});
```

Why Different Values for CI and Local?

Local Machine → more workers

Your personal computer is powerful

You have full resources available

More workers = faster test runs

workers: 4

CI/CD Pipeline → fewer workers

CI machines are shared and have limited resources

Too many workers can crash or slow down the machine

Fewer workers = more stable pipeline

workers: 2

Eg: workers: process.env.CI ? 1 : undefined

```
//      ↑      ↑      ↑
//      condition yes  no
```

Are we running on a CI/CD pipeline?? 1 If YES — run only 1 test at a time: undefined If NO — let Playwright decide automatically

In Plain English

"On CI/CD pipeline run one test at a time. On local machine let Playwright figure out the best number automatically."

6. reporters- Controls how Playwright shows you the test results — in the terminal, as an HTML page, as a file etc.

```
Eg: export default defineConfig({
  reporter: 'html', //show results as a nice HTML report
});
```

Eg: Using Multiple Reporters at Once

You can combine reporters so results go to multiple places at the same time:

```
export default defineConfig({
  reporter: [
```

```

    ['list'], // show in terminal
    ['html', { outputFolder: 'my-report' }], // also save HTML report
    ['json', { outputFile: 'results.json' }], // also save JSON file
  ],
});

```

7. Timeout- Sets the maximum time a single test is allowed to run before Playwright gives up and marks it as failed.

Eg: export default defineConfig({
timeout: 30000, // each test has 30 seconds to complete
});

If you don't set timeout, Playwright defaults to 30 seconds per test.

What Happens When it Times Out

```

export default defineConfig({
  timeout: 5000, // 5 seconds
});

```

Test starts running...

1 second → still running

2 seconds → still running

3 seconds → still running

4 seconds → still running

5 seconds →  TIMEOUT!

Error: Test timeout of 5000ms exceeded

8. expect with {timeout}: sets a global timeout for expect assertions:

Eg: import { defineConfig } from '@playwright/test';

```

export default defineConfig({
  expect: {
    timeout: 5000, // 5 seconds
  },
});

```

9. Use with {trace, headless}- sets the global browser behavior that applies to all your tests.

- headless controls whether you see the browser open when tests run.

true=>Browser runs invisibly in the background — you see nothing

false =>A real browser window pops open and you can watch the test click around

- A trace is a flight recorder for your tests — it captures a full recording of everything that happened: screenshots, network requests, DOM snapshots, console logs, and a timeline of actions.

'off'=>'Never record anything

'on'=>'Always record, every test

'on-first-retry'=>Only record when a test fails and retries

'retain-on-failure'=>Record always, but delete the trace if the test passes

Why 'on-first-retry' is the sweet spot:

You don't waste disk space recording passing tests

The moment something fails, you get the full recording to debug with

You open it with `npx playwright show-trace trace.zip` and get a visual, step-by-step playback — like a DVR for your test

Eg: `export default defineConfig({`

```
  use: {  
    headless: true,  
    trace: 'on-first-retry',  
  },  
});
```

10. Projects with name and use- allows you to run your tests multiple times across different configurations — most commonly different browsers, but also different devices, viewports, or setups.

-name

Just a label for the project. It shows up in your test results so you know which browser/config a test ran on or failed in.

✓ [chromium] › login.spec.ts

✓ [firefox] › login.spec.ts

✗ [Mobile Safari] › login.spec.ts ← easy to spot which one failed

You can also use it to run only one project from the command line:

`npx playwright test --project=chromium`

-use (inside a project)

This is the same use block as the global one, but it only applies to that specific project. It overrides or extends the global use settings for that project alone.

```
projects: [  
  {  
    name: 'chromium',  
    use: {  
      browserName: 'chromium',  
      viewport: { width: 1280, height: 720 },  
    },  
  },  
  {  
    name: 'Mobile Chrome',  
    use: {  
      ...devices['Pixel 7'], // pre-built device profile  
    },  
  },  
]
```

devices['Pixel 7'] is a shorthand that Playwright provides — it automatically sets the correct viewport, user agent, and touch settings for that device.

The big picture

Think of projects like test lanes on a highway. Every test you write drives down each lane:

Your tests ———▶ **[Chromium lane]** → runs all tests on Chrome
 ├──▶ **[Firefox lane]** → runs all tests on Firefox
 └──▶ **[Mobile Safari lane]** → runs all tests on iPhone

One test file, written once, gets automatically executed in every project you define.

- **HTML Report Generation-** produces a self-contained folder that contains report for the test run and can be served as a webpage-shows a report of all your tests that have been run.

-Also shows which browser the test ran-how long the test took to run.

-Test results can be categorised into passed tests, failed tests, flakey tests and skipped tests

- You can also search for a particular test-clicking on a test will open a detailed view of the test consisting of more information on your tests such as the errors and test steps
- You can also embed screenshots, videos and trace viewers into the HTML report-these configurations can be specified in the **Playwright.config.ts** file
- Retry details can also be viewed in HTML report

`npm playwright test --reporter=html=>` command for running and generating HTML test reports of your tests

`npm playwright show-report=>` command for opening the generated HTML report of your tests

`npm playwright show-report <custom folder name>=>` command for opening the generated HTML report of your tests in a custom folder of your choice

- **Playwright Test Annotations (Typescript)**- Playwright supports test annotations to deal with failures, flakiness, skip, focus,etc. Test annotations are labels you stick onto a test to change how it behaves or to give it extra information — without changing the actual test logic.
Think of them like sticky notes on a test.

1. `test.describe()`- declares a group of tests-Group tests together-"These tests all belong to the same topic-keep them together." Just an organiser — like putting tests in a folder.

Eg: `test.describe('Login Page', () => {
 test('valid login', async ({ page }) => { });
 test('invalid login', async ({ page }) => { });
});`

2. `test.only()`- declares a focused test-if there are some focused tests, all of them will run but nothing else-Run ONLY this test-"Ignore all other tests — just run this one." Used when debugging a specific test and you don't want to wait for all others.

Eg: `test.only('should login', async ({ page }) => {
 // only this test runs, everything else is ignored
});`

3. **test.skip()**- marks the test as irrelevant. Playwright does not run such a test-Skip this test-**"Don't run this test right now — I'll come back to it later."** Used when a feature is broken or not built yet. (use this annotation when the test is not applicable in some configuration)

Eg: `test.skip('should load dashboard', async ({ page }) => {
 // this test will not run at all
});`

(or)

```
test("Practice Test 5", async ({ page }) => {  
  test.skip()  
  console.log("Starting Practice Test5");  
  console.log("Ending Practice Test 5");  
})
```

(or) we can even skip a particular browser also like so

```
test("Practice Test 5", async ({ page, browserName }) => {  
  test.skip(browserName === "chromium", "Skipping test on Chromium");  
  console.log("Starting Practice Test5");  
  console.log("Ending Practice Test 5");  
}) //skips only the chrome browser in this case but executes the  
firefox and webkit browser in this test
```

4. **test.fixme()**- declares a test to be fixed-this test won't run-This needs fixing-**"This test is broken and needs to be fixed — skip it for now but don't forget it."** Like skip but with a stronger signal that it's a TODO.

Eg: `test.fixme('should submit form', async ({ page }) => {
 // test is skipped with a signal that it needs attention
});`

5. test.slow()- marks the test as slow and triples the test timeout-Give this test extra time-"**This test takes longer than usual — be patient with it.**" Playwright triples the default timeout automatically.

```
Eg: test('should upload large file', async ({ page }) => {  
    test.slow()  
    // timeout is automatically tripled for this test  
});
```

6. test.setTimeout()- changes the timeout for the currently running test. Zero means no timeout-sets a maximum time limit in ms for the test to finish executing-"**give this test X milliseconds to finish, or give up and mark it as failed.**"By default, Playwright gives each test 30 seconds. But sometimes:

A test is doing something slow (big file upload, slow network, heavy page)

You want to be stricter and fail faster

You want to give more time for a specific test

```
Eg: test('slow test', async ({ page }) => {  
    test.setTimeout(60000); // Give this test 60 seconds instead of 30  
    await page.goto('https://slow-website.com');  
    // ... rest of test  
});
```

(or) set it globally

```
// in playwright.config.ts  
export default defineConfig({  
    timeout: 60000, // 60 seconds for every test  
});
```

7. test.fail()- unconditionally marks a test as "should fail". Playwright runs this test and ensures that it is actually failing. This is useful for documentation purposes to acknowledge that some functionality is broken until its fixed-It's commonly used to document known bugs without breaking your entire test pipeline-"**I know this test is broken right now, don't panic when it fails.**"

```
Eg: test("button should submit form", async ({ page }) => {
```

```
test.fail(); // I know this is broken, I expect it to fail
await page.click("#submit-button");
await expect(page).toHaveURL("/success");
});
```

- **Playwright Hooks-(beforeEach, afterEach, beforeAll, afterAll)**

Think of hooks like setup and cleanup crew at an event.

Before a concert starts, the crew sets up the stage. After it ends, they clean everything up. The performers (tests) just walk in and do their job — they don't worry about setup or cleanup.

That's exactly what hooks do in Playwright.

Note: set `fullyparallel` configuration in `playwright.config.ts` file as `false`

1. **beforeEach-** Runs before every single test in a test file regardless of where its placed-when called inside `test.describe()`, it runs before each test in the group-if multiple `beforeEach` hooks are added it runs in the order they are written in the file- Like a waiter **resetting your table** before every new customer sits down.

Syntax:

`test.beforeEach(anonymous hookFunction)`

`test.beforeEach(title of test (optional), hookFunction)`

**Eg: `test.beforeEach(async ({ page }) => {`
 `await page.goto("https://myapp.com/login");`
 `// e.g. log in before every test`
`});`**

2. **afterEach-** Runs after every single test in a test file regardless of where its placed-when called inside `test.describe()`, it runs after each test in the group-if multiple `afterEach` hooks are added it runs in the order they are written in the file- Like a waiter **resetting your table** after every new customer sits down.

Syntax:

`test.afterEach(anonymous hookFunction)`

test.afterEach(title of test (optional), hookFunction)

**Eg: test.afterEach(async ({ page }) => {
 console.log("Cleaning up after each test");
 // e.g. log out, clear cookies
});**

- 3. beforeAll-** runs once before all tests, not before each one-when called inside `test.describe()`, it runs before all the tests in the group-if multiple `beforeAll` hooks are added it runs in the order they are written in the file-**Think of it like a one time setup before a show begins. Before a movie starts in a theater, the staff sets up the projector, adjusts the sound, and dims the lights — once. They don't do this before every single scene.**

Syntax:

test.beforeAll(anonymous hookFunction)

test.beforeAll(title of test (optional), hookFunction)

**Eg: test.beforeAll(async () => {
 console.log("I run only once before all tests");
});**

**test("Test 1", async ({ page }) => {
 console.log("Test 1");
});**

**test("Test 2", async ({ page }) => {
 console.log("Test 2");
});**

**test("Test 3", async ({ page }) => {
 console.log("Test 3");
});**

- 4. afterAll-** runs once after all tests, not after each one-when called inside `test.describe()`, it runs after all the tests in the group-if multiple `afterAll` hooks are added it runs in the order they are written in the file-**Think of it like the cleanup crew after a concert ends. After the show is completely over,**

the crew comes in once to pack up the stage, turn off the lights, and lock the doors. They don't clean up after every single song.

Syntax:

`test.beforeAll(anonymous hookFunction)`

`test.beforeAll(title of test (optional), hookFunction)`

Eg: `test.afterAll(async () => {
 console.log("I run only once after all tests are done");
});`

`test("Test 1", async ({ page }) => {
 console.log("Test 1");
});`

`test("Test 2", async ({ page }) => {
 console.log("Test 2");
});`

`test("Test 3", async ({ page }) => {
 console.log("Test 3");
});`

Using page Fixture with Hooks

Hook	Can use page?
<code>beforeEach</code>	✓ Yes
<code>afterEach</code>	✓ Yes
<code>beforeAll</code>	✗ No
<code>afterAll</code>	✗ No

Why?

page represents a fresh browser tab created for each test.

Think of page like a disposable cup — a new one is created for every test and thrown away after.

beforeEach and afterEach run around each test, so the cup exists at that point



beforeAll and afterAll run outside of any test, so there's no cup yet/anymore ❌

beforeEach and afterEach ✔️

typescript

```
test.beforeEach(async ({ page }) => {  
  await page.goto("https://www.google.com"); // ✔️ works perfectly  
});
```

```
test.afterEach(async ({ page }) => {  
  console.log(await page.title()); // ✔️ works perfectly  
});
```

beforeAll and afterAll ❌

typescript

```
test.beforeAll(async ({ page }) => {  
  await page.goto("https://www.google.com"); // ❌ throws error  
});
```

```
test.afterAll(async ({ page }) => {  
  console.log(await page.title()); // ❌ throws error  
});
```

Error you'll see:

"page" fixture is not supported for beforeAll/afterAll.
Use browser fixture instead.

✔️ Workaround for beforeAll and afterAll:

Use browser fixture to manually create a page:

typescript

```
import { test, Browser, Page } from "@playwright/test";
```

```
let page: Page;
```

```
test.beforeAll(async ({ browser }) => {  
  page = await browser.newPage(); // manually create a page  
  await page.goto("https://www.google.com");  
});
```

```
test.afterAll(async () => {  
  await page.close(); // manually close the page  
});
```

Bottom line:

page only exists during a test. Hooks that run outside a test (**beforeAll**, **afterAll**) can't use it directly — you have to create it manually using **browser**.

Eg: import { test, Browser, Page } from "@playwright/test";

```
let page: Page;
```

```
//  Using browser fixture to manually create page  
test.beforeAll(async ({ browser }) => {  
  page = await browser.newPage();  
  await page.goto("https://www.saucedemo.com/");  
  console.log("beforeAll - page created and navigated once");  
});
```

```
//  page fixture works directly here  
test.beforeEach(async ({ page }) => {  
  console.log("beforeEach - runs before every test");  
});
```

```
//  page fixture works directly here  
test.afterEach(async ({ page }) => {  
  console.log("afterEach - runs after every test");  
});
```

```
//  Using manually created page to close
```

```
test.afterAll(async () => {  
  await page.close();  
  console.log("afterAll - page closed, cleaning up");  
});
```

```
test("Test 1", async ({ page }) => {  
  console.log("Test 1 running");  
  await page.goto("https://www.saucedemo.com/");  
  console.log(await page.title());  
});
```

```
test("Test 2", async ({ page }) => {  
  console.log("Test 2 running");  
  await page.goto("https://www.saucedemo.com/");  
  console.log(await page.title());  
});
```

```
test("Test 3", async ({ page }) => {  
  console.log("Test 3 running");  
  await page.goto("https://www.saucedemo.com/");  
  console.log(await page.title());  
});
```

Output:

```
// beforeAll runs once  
beforeAll - page created and navigated once
```

```
// Test 1 cycle  
beforeEach - runs before every test  
Test 1 running  
Swag Labs  
afterEach - runs after every test
```

```
// Test 2 cycle  
beforeEach - runs before every test  
Test 2 running  
Swag Labs  
afterEach - runs after every test
```

```
// Test 3 cycle  
beforeEach - runs before every test
```


Test 3 running

Swag Labs

afterEach - runs after every test

// afterAll runs once

afterAll - page closed, cleaning up

Key things to notice:

Thing	Explanation
<code>let page: Page</code>	Declared outside so all hooks can access it
<code>beforeAll</code> uses <code>{ browser }</code>	Manually creates a page since <code>{ page }</code> not allowed
<code>beforeEach</code> uses <code>{ page }</code>	Works fine since it runs around each test
<code>afterEach</code> uses <code>{ page }</code>	Works fine since it runs around each test
<code>afterAll</code> fixture uses no	Uses the manually created <code>page</code> variable to close it

- Playwright fill(), pressSequentially() and press() methods

1. **locator.fill()**- easiest way to fill out the form fields-focuses-It's simply typing text into an input field on a webpage-works like its pasting text instantly into the fields-It's one of the most commonly used commands in any login or form test-works for <input>, <textarea> and content editable elements

Eg: `test('Fill, PressSequentially, Press Test', async ({page}) => {
 await page.goto("https://ultimateqa.com/filling-out-forms/")
 await page.locator("#et_pb_contact_name_0").fill("Amrutha Sriram")
 await page.locator("#et_pb_contact_message_0").fill("This is a test message")
})`

```

                                                                    await
page.goto("https://www.w3schools.com/tags/tryit.asp?filename=tryhtml
5_global_contenteditable");

                                                                    await
page.frameLocator('iframe[name="iframeResult"]').getByText('This is a
paragraph. It is').fill("This is a test message");

})

```

2. **locator.pressSequentially**- types into the field character by character as if it were a real user

```

Eg: test('Press methodTest', async ({page}) => {
  await page.goto("https://www.google.com/")
  //await page.locator("#APjFqb").pressSequentially("Playwright");

```

```

  await
  page.locator("#APjFqb").pressSequentially("Playwright",{delay:200})//wi
  th delay between each character

```

```

})

```

3. **locate.press()**- hits one specific key — like pressing Enter, Tab, Escape, etc.

```

  Eg: test('Press method Test', async ({page}) => {
    await page.goto("https://www.google.com/")
    await page.locator("#APjFqb").pressSequentially("Playwright",{delay:200})
                                                                    await
    page.locator("#APjFqb").press("ArrowDown+ArrowDown+ArrowDown")
    await page.locator("#APjFqb").press("Enter")
    await page.locator("#APjFqb").press("Backspace")
  })

```

Note: **fill()** types a whole word.

pressSequentially() types one character at a time.

.press() hits one specific key — like pressing Enter, Tab, Escape, etc.

- **Playwright Different Click Methods**-Click, Double Click, Right Click and Programmatic Click

1. **Click()**- It's simply clicking on anything on a webpage — just like you would with a mouse. Whatever you can click with your mouse, click() does it automatically for you.

Syntax: locator.click()

Eg: // Click a button

```
await page.locator("#login-button").click();
```

// Click a link

```
await page.locator("a[href='/about']").click();
```

// Click a checkbox

```
await page.locator("#terms-checkbox").click();
```

// Click a dropdown

```
await page.locator("#dropdown").click();
```

// Click a radio button

```
await page.locator("#male-radio").click();
```

Eg: In VSCode:

```
test("Login Test using Click method", async ({ page }) => {  
  await page.goto("https://www.saucedemo.com/");
```

```
  await page.locator("#user-name").fill("standard_user");  
  await page.locator("#password").fill("secret_sauce");
```

```
  // Click the login button
```

```
  await page.locator("#login-button").click();  
});
```

2. **Double Click**- dblclick() method performs a simple double click on an element-It's simply clicking twice rapidly on an element — just like double clicking with your mouse. Single click usually selects something. Double click usually opens or edits something.

Syntax: locator.dblclick()

Eg: When would you use double click:

```
// Double click to edit a text field inline
await page.locator(".editable-text").dblclick();
```

```
// Double click to open a file/folder
await page.locator(".folder-item").dblclick();
```

```
// Double click to select a word in a text area
await page.locator("textarea").dblclick();
```

```
// Double click a canvas element in a drawing app
await page.locator("#canvas").dblclick();
```

Eg: In VSCode:

```
test("Double Click Test", async ({ page }) => {

  await page.goto("https://the-internet.herokuapp.com/add_remove_elements/");

  // Double click on an element
  await page.getByRole('button', { name: 'Add Element' }).dblclick()
});
```

- 3. Right Click/Context Click-** Right clicking usually opens a context menu on a webpage — just like right clicking with your mouse opens options like copy, paste, open in new tab etc- `click({button:'right'})` method performs a simple right click on an element

Under the hood Playwright will:
wait for element in given selector to be in DOM
Scroll the element into view

Syntax: `locator.click({button:'right'})`

Eg: Common use cases:

```
// Right click to open context menu
await page.locator("#element").click({ button: "right" });
```

```
// Right click on an image
await page.locator("img").click({ button: "right" });
```

```
// Right click on a link
```

```
await page.locator("a").click({ button: "right" });
```

```
// Right click on a table row
```

```
await page.locator("tr").first().click({ button: "right" });
```

Eg: In VSCode:

```
test("Right Click Test", async ({ page }) => {  
  await page.goto("https://swisnl.github.io/jQuery-contextMenu/demo.html")  
  //await page.getByText('right click me', { exact: true }).click({ button: 'right' })  
  await page.getByRole('heading', { name: 'Example code: Simple Context'  
}).click({button:'right'})  
})
```

4. **Programmatic Click-** If you're not interested in testing your app under the real conditions and want to simulate the click by any means possible, you can trigger the `HTMLElement.click()` behaviour via simply dispatching a click event on the element with `locator.dispatchEvent.click()`- **Think of it like sending a click signal directly to an element without even involving the mouse at all. Normal click() — Playwright moves a virtual mouse to the element and clicks. Programmatic click — Tells browser via JavaScript to click. `dispatchEvent('click')` — Manually fires the click event directly on the element.**

Syntax: `await page.locator("selector").dispatchEvent("click");`

When to use **dispatchEvent**:

Situation	Use
Element is hidden	✓ <code>dispatchEvent</code> bypasses visibility
Element is covered	✓ <code>dispatchEvent</code> bypasses obstruction
Need to trigger non-click events	✓ Only <code>dispatchEvent</code> can fire mouseover, blur, focus etc
Normal visible element	✗ Just use <code>.click()</code>

`dispatchEvent("click")` is the most direct way to fire any event on an element. It skips all checks and just fires the event signal. The biggest advantage over other methods is that it can fire any event — not just clicks — making it useful for triggering hover, focus, blur, and keyboard events too.

Eg: // Mouse events

```
await page.locator("#element").dispatchEvent("click");
await page.locator("#element").dispatchEvent("dblclick");
await page.locator("#element").dispatchEvent("mouseover");
await page.locator("#element").dispatchEvent("mousedown");
await page.locator("#element").dispatchEvent("mouseup");
```

// Keyboard events

```
await page.locator("#input").dispatchEvent("keydown");
await page.locator("#input").dispatchEvent("keyup");
```

// Focus events

```
await page.locator("#input").dispatchEvent("focus");
await page.locator("#input").dispatchEvent("blur");
```

// Input events

```
await page.locator("#input").dispatchEvent("input");
await page.locator("#input").dispatchEvent("change");
```

Eg: In VSCode:

```
test("DispatchEvent Click Test", async ({ page }) => {
  await page.goto("https://www.saucedemo.com/");

  await page.locator("#user-name").fill("standard_user");
  await page.locator("#password").fill("secret_sauce");

  // Dispatch click event on login button
  await page.locator("#login-button").dispatchEvent("click");
});
```

- **Playwright-Handling radio button-**

1. We need to use the `locator.check()` method to check the radio button with `input[type='radio']` elements

2. To verify if the radio button is checked/selected or not we need to use the **toBeChecked()** method with expect like so:
await expect(locator).toBeChecked()

Eg: `test("Handling Radio Buttons", async ({ page }) => {
 await page.goto("https://artoftesting.com/samplesiteforselenium")
 const femaleRadioButton = await page.locator('#female')
 const maleRadioButton = await page.locator('#male')
 await femaleRadioButton.check()
 await expect(femaleRadioButton).toBeChecked()
 await expect(maleRadioButton).not.toBeChecked()
})`

- **Playwright-Handling Checkbox-**

1. We need to use the **locator.check()** method to check the checkbox with **input[type='checkbox']** and **[role=checkbox]** elements
2. To verify if the checkbox is checked/selected or not we need to use the **toBeChecked()** method with expect like so:
await expect(locator).toBeChecked()
3. We need to use the **.unchecked()** method to uncheck the checkbox

Eg: `test("Handling checkboxes", async ({page}) => {
 await page.goto("https://artoftesting.com/samplesiteforselenium")
 await page.locator(".Automation").check()
 await expect(page.locator(".Automation")).toBeChecked()
 await page.locator(".Performance").check()
 await page.locator(".Performance").unchecked();
 await expect(page.locator(".Performance")).not.toBeChecked();

 if(await page.locator(".Automation").isChecked()){
 console.log("Element was checked");
 await page.locator(".Automation").unchecked();
 }
})`

- **Playwright-Handling dropdowns-Selecting a single option**

1. We can select one or multiple options with **locator.selectOption()**
2. You can specify **option, value, label or index** to select

Syntax:

```
=>locator.selectOption(<value/label/visible text>)
=>locator.selectOption({value: value attribute value})
=>locator.selectOption({label:label attribute value})-incase label
attribute isn't there visible text can be provided as the label
attribute value but if label attribute is there then providing visible
text won't work like {label: visible text}
=>locator.selectOption({index:index value of option(starts from
0)})
```

Eg: test("Handle Select Dropdown with Value attribute and Visible text",async({page})=>{

```
    await
    page.goto("https://artoftesting.com/samplesiteforselenium")
    await page.locator("#testingDropdown").selectOption("Manual")
    //selecting option by value attribute's value
    await
    page.locator("#testingDropdown").selectOption("Database
    Testing") //selecting option by visible text
    await
    page.locator("#testingDropdown").selectOption({value:'Performan
    ce'}) //selecting option with object=>{value:value attribute's value}
    await
    page.locator("#testingDropdown").selectOption({label:'Database
    Testing'}) //selecting option with object=>{label:visible text} as
    label attribute isn't there
    await
    page.locator("#testingDropdown").selectOption({index:2});
    })
```

test("Handle Select Dropdown with Label attribute",async({page})=>{

```
    await
    page.goto("https://www.w3schools.com/tags/tryit.asp?filename=tr
    yhtml_option_label");
    await
    page.frameLocator('iframe[name="iframeResult"]').getByLabel('Ch
    oose a car:').selectOption('Mercedes') //selecting option by label
    attribute value
    await
    page.frameLocator('iframe[name="iframeResult"]').getByLabel('Ch
```



```

oose a car:').selectOption({label:'Audi'}) //selecting option with
object=>{label:label attribute's value}
//await
page.frameLocator('iframe[name="iframeResult"]').getByLabel('Ch
oose a car:').selectOption({label:'Audi (Auto Union Deutschland
Ingolstadt)')) //selecting option with object=>{label:visible text}>it
will fail as label attribute is present

})

```

- **Playwright-Handling dropdowns-Selecting a multiple options**

1. We can select multiple options from the multi select dropdown using **locator.selectOption()**
2. You can specify options by giving value, label or visible text within an array to select desired options

Syntax:

```
locator.selectOption(['item 1', 'item 2', 'item 3'])
```

Eg: test("Handle Multi Select Dropdown",async({page})=>{
 await page.goto("https://demoqa.com/select-menu");
 await page.locator("#cars").selectOption('opel'); //selecting options
 by providing value
 //await page.locator("#cars").selectOption(['Volvo', 'Opel','Audi']) //selecting
 options by providing an array of visible text
 })

- **Playwright-Handling Javascript Alerts, Dialog boxes, Prompts**

By default, auto dialogs are dismissed by Playwright so you won't have to handle them-you can register a dialog handler before the action that triggers the dialog to either `dialog.accept()` or `dialog.dismiss()`-When there is a `page.on('dialog')` listener present it must either `dialog.accept()` or `dialog.dismiss()` the dialog- otherwise the page will freeze waiting for the dialog, and therefore it will block further page execution until they are handled-there are 3 types of dialog boxes: **alert, confirm and prompt**

Eg: import {test,expect} from "@playwright/test";

```
test("Handle alert Dialogs",async({page})=>{

    await page.goto("https://the-internet.herokuapp.com/javascript_alerts");

    page.on("dialog", dialog =>{
        expect(dialog.type()).toEqual("alert");
        expect(dialog.message()).toEqual("I am a JS Alert");
        dialog.accept(); //handle the alert by accepting it so that it will close
    })

    await page.getByRole('button', { name: 'Click for JS Alert' }).click()
    await expect(page.locator("#result")).toHaveText("You successfully clicked an alert") //provide assertion

})

test("Handle Confirm Dialog", async({page})=>{
    await page.goto("https://the-internet.herokuapp.com/javascript_alerts");

    page.on("dialog", dialog =>{
        expect(dialog.type()).toEqual("confirm");
        expect(dialog.message()).toEqual("I am a JS Confirm");
        dialog.dismiss(); //handle the alert by dismissing it so that it will close
    })

    await page.getByRole('button', { name: 'Click for JS Confirm' }).click();

})

test("Handle Prompt Dialog", async({page})=>{
    await page.goto("https://the-internet.herokuapp.com/javascript_alerts");

    page.on("dialog", dialog =>{
        expect(dialog.type()).toEqual("prompt");
        expect(dialog.message()).toEqual("I am a JS prompt");
        expect(dialog.defaultValue()).toEqual("");
        dialog.accept("abc"); //handle the alert by providing input and accepting it
    so that it will close
        //dialog.dismiss(); //handle the alert by dismissing it so that it will close
    })

    await page.getByRole('button', { name: 'Click for JS Prompt' }).click();
```

```
})
```

Note: `.on()` in Playwright — Layman's Terms

Think of it like setting up a security camera that watches for something to happen.

You don't know exactly when something will happen, but when it does — you want to react to it automatically.

That's `.on()` — it listens for an event and runs your code when that event happens.

Basic syntax:

typescript

```
page.on("eventName", callbackFunction);
```

Real world example:

typescript

```
test("Listen for Console Messages", async ({ page }) => {  
  // Set up listener BEFORE navigating  
  page.on("console", message => {  
    console.log("Browser said:", message.text());  
  });  
  
  await page.goto("https://www.saucedemo.com/");  
});
```

Most commonly used events:

typescript

```
// 1. Listen for console messages  
page.on("console", message => {  
  console.log("Console:", message.text());  
});  
  
// 2. Listen for page errors  
page.on("pageerror", error => {  
  console.log("Page error:", error.message);  
});
```

```

});

// 3. Listen for network requests
page.on("request", request => {
    console.log("Request made:", request.url());
});

// 4. Listen for network responses
page.on("response", response => {
    console.log("Response received:", response.url(), response.status());
});

// 5. Listen for new tabs/popups opening
page.on("popup", popup => {
    console.log("New popup opened:", popup.url());
});

// 6. Listen for dialogs (alert, confirm, prompt)
page.on("dialog", async dialog => {
    console.log("Dialog message:", dialog.message());
    await dialog.accept(); // click OK
});

// 7. Listen for page navigation
page.on("load", () => {
    console.log("Page fully loaded");
});

// 8. Listen for page crashes
page.on("crash", () => {
    console.log("Page crashed!");
});

```

Full real world example:

typescript

```

test("Monitor Everything", async ({ page }) => {

    // Watch for console messages
    page.on("console", message => {
        console.log("Console:", message.text());
    });

    // Watch for errors

```

```

page.on("pageerror", error => {
  console.log("Error:", error.message);
});

// Watch for network requests
page.on("request", request => {
  console.log("Request:", request.url());
});

// Watch for responses
page.on("response", response => {
  console.log("Response:", response.status(), response.url());
});

// Now navigate — all listeners are active
await page.goto("https://www.saucedemo.com/");
await page.locator("#user-name").fill("standard_user");
await page.locator("#password").fill("secret_sauce");
await page.locator("#login-button").click();
});

```

⚠ Important — always set up `.on()` BEFORE the action:

typescript

// ❌ Wrong - listener set up AFTER navigation, will miss events

```

await page.goto("https://www.saucedemo.com/");
page.on("console", message => {
  console.log(message.text());
});

```

// ✅ Correct - listener set up BEFORE navigation, catches all events

```

page.on("console", message => {
  console.log(message.text());
});
await page.goto("https://www.saucedemo.com/");

```

All common events at a glance:

Event	When it fires
<code>console</code>	Browser logs something
<code>e</code>	

pageerror	JavaScript error on page
request	Network request is made
response	Network response is received
popup	New tab or window opens
dialog	Alert, confirm, or prompt appears
load	Page finishes loading
crash	Page crashes

Bottom line:

.on() is like a watchdog — you set it up once and it keeps watching in the background. Whenever the event you're watching for happens, your code runs automatically. Always set it up before the action you want to monitor.

- **Playwright-Handling iframes-** used to embed another document within the current HTML document- The below methods can be used to access the iframe in Playwright:

page.frame(<name/url>) => name attribute or url

page.frameLocator(<selector value>) => locator like Xpath, CSS Selector, etc

In VSCode:

```
Eg: test("Handle Iframe with name attribute", async({page})=>{
  await
  page.goto("https://www.w3schools.com/html/tryit.asp?filename=tryhtml5_input_form");
  const w3Frame = page.frame("iframeResult"); //assigning the iframe component to a variable using name attribute
  await w3Frame?.locator("#fname").fill("Amrutha");
})
```

```
test("Handle Iframe with URL", async({page})=>{
  await page.goto("https://www.w3schools.com/html/html_iframe.asp");
  const w3Frame2 =
page.frame({url:"https://www.w3schools.com/html/default.asp"});
//assigning the iframe component to a variable using URL of iframe
  await w3Frame2?.getByLabel('Button to open search field').click();
//assign the location of element obtained by locator starting from the
.getBy part of the location
  await w3Frame2?.getByPlaceholder('Search...').fill("Typescript");

}) //url can be obtained by right-clicking on the frame element on the
webpage
```

```
test("Handle Iframe with frameLocator Method", async({page})=>{
  await page.goto("https://www.w3schools.com/html/html_iframe.asp");
  const w3Frame3 = page.frameLocator("[title='W3Schools HTML
Tutorial']"); //assigning the iframe component to a variable using
frameLocator method and title attribute of the frame element
  await w3Frame3.getByLabel('Button to open search field').click();
  await w3Frame3.getByPlaceholder('Search...').fill("Playwright");
```

(or)

```
const w3Frame3= page.locator('iframe[title="W3Schools HTML
Tutorial"]').contentFrame().getByRole('button', { name: 'Button to open
search field' }) //locating element and assigning to a variable
  await w3Frame3.click(); //Performing the action on that element
  const w3Frame4=page.locator('iframe[title="W3Schools HTML
Tutorial"]').contentFrame().getByRole('textbox', { name: 'Search field' })
//locating element and assigning to a variable

  await w3Frame4.fill("Javascript"); //Performing the action on that
element

}) //this method is easier
```

Note: The **?** — Optional Chaining (use this only if we pass name/url to the .frame method)

The **?** is called the **optional chaining operator**. It means:

"Only call `.locator()` if `w3Frame` is **not** `null` or `undefined`. If it is, just return `undefined` instead of throwing an error."

Without ? vs With ?

typescript

// ❌ Without ? — CRASHES if w3Frame is null/undefined

`await w3Frame.locator("#fname").fill("Amrutha");`

// TypeError: Cannot read properties of undefined

// ✅ With ? — SAFELY skips if w3Frame is null/undefined

`await w3Frame?.locator("#fname").fill("Amrutha");`

// Just does nothing, no crash

Layman Terms

Think of it like asking someone for directions:

- **Without ?**: You assume someone is standing there. If no one is there, you crash into a wall.
 - **With ?**: You check if someone is there first. If not, you just quietly move on.
-

Why does `w3Frame` need ? here?

Because `frame()` in Playwright can return `null` if the frame isn't found:

typescript

// `frame()` returns `Frame | null`

`const w3Frame = page.frame({ name: "w3frame" }); // might be null!`

`await w3Frame?.locator("#fname").fill("Amrutha"); // safe ✅`

An **iframe** (inline frame) is an HTML element that lets you embed another webpage inside your current webpage — like a window within a window.

html

`<iframe src="https://example.com" width="600" height="400"></iframe>`

Layman's Terms

Imagine you're reading a newspaper, and there's a **cut-out section** where a completely different, live TV screen is playing. You can see the TV content right there on the newspaper page, but it's coming from a totally separate source.

That's an iframe — a **"frame within a page"** that displays content from somewhere else.

Real-world examples you've already seen

- **YouTube videos embedded** on a blog post → that video player is an iframe
 - **Google Maps** embedded on a restaurant's website → iframe
 - **Payment forms** (like Stripe checkout) on shopping sites → iframe
 - **Social media like/share buttons** → often iframes
-

Key things to know

It's self-contained — the content inside an iframe has its own HTML, CSS, and JavaScript. It doesn't mess with the parent page.

It can load external content — the src can point to any URL, not just your own site.

Security matters — websites can block themselves from being embedded in iframes (to prevent clickjacking attacks). You've probably seen broken iframes that just show an error for this reason.

It has limitations — you generally can't access or manipulate the content inside an iframe if it's from a different domain (this is called the **same-origin policy**).

In short: an iframe is just a **webpage inside a webpage**.

iframes in Playwright

Playwright has special methods to interact with iframes since they're separate documents embedded within a page.

Locating an iframe

Method 1: `frameLocator()` (Recommended)

javascript

// By CSS selector

```
const iframe = page.frameLocator('iframe');
```

```
const iframe = page.frameLocator('#my-iframe');
```

```
const iframe = page.frameLocator('iframe[name="payment"]');
```

// Then interact with elements inside it

```
await iframe.locator('input[name="email"]').fill('test@example.com');
```

```
await iframe.locator('button[type="submit"]').click();
```

Method 2: `frame()` — by name or URL

javascript

// By name attribute

```
const frame = page.frame({ name: 'my-frame' });
```

// By URL

```
const frame = page.frame({ url: /example\.com/ });
```

// Then interact

```
await frame.click('button');
```

```
await frame.fill('input', 'hello');
```

- **Playwright-Handling new page/new tab/child windows**-an isolated `BrowserContext` instance is created for each test-since contexts are isolated between each other, every test gets a fresh environment, even when multiple tests run in a single browser-context won't share cookies/cache with other browser contexts-the `page` event on browser contexts can be used to get new pages that are created in the context-this can be used to handle new pages opened by `target="_blank"` (target is the html attribute) links-

Imagine you're browsing and you click a link that opens a brand new tab. Your original tab is still there, but now there's a new one.

Playwright by default only watches the original tab. If a new tab opens, Playwright has no idea what's happening there — unless you tell it to watch for it- **How Playwright handles it: When a click opens a new tab or window, Playwright can capture it using the `context.waitForEvent('page')` method.**

```
//start waiting for new page before clicking
const pagePromise = context.waitForEvent('page');
// Sets up a LISTENER on the browser context BEFORE any click happens
// context.waitForEvent('page') returns a PROMISE — it doesn't await yet
// Think of it like setting up a camera to record BEFORE the action happens
// If you set this up AFTER the click, you might miss the new tab opening
```

```
await page.getByText('open new tab').click();
// Clicks the element that contains the text 'open new tab'
// This action triggers a new tab/window to open in the browser
// The listener we set up above is now actively watching for this new tab
```

```
const newPage = await pagePromise;
// NOW we await the promise — meaning we pause here and WAIT
// until the new tab actually opens
// Once the new tab opens, it gets assigned to 'newPage' variable
// 'newPage' is now a separate Page object representing the new tab
// We can use it just like we use 'page' to interact with elements
```

```
await newPage.waitForLoadState();
//use the above statement incase page is taking time to load
// Waits for the new tab to fully load before we do anything
// Default load state is 'load' — waits for all resources to load
// Without this, we might try to interact with elements that aren't ready yet
// Other options:
// 'domcontentloaded' — faster, waits only for HTML to load
// 'networkidle' — slower, waits until no network activity
```

```
console.log(await newPage.title());
// Fetches the TITLE of the newly opened tab
// newPage.title() returns a promise so we need to await it
// Prints the page title to the terminal/console
// Useful for:
```

```
//  Debugging — confirms the right page opened
//  Verification — checking if navigation was successful
```

Breaking Down Each Statement

1. "An isolated BrowserContext is created for each test"

Think of it like this:

```
Browser (the actual Chrome/Firefox app)
├── BrowserContext (like a fresh incognito window)
│   └── Page (a tab inside that window)
typescript
test('test 1', async ({ page }) => { ... }); // gets its OWN context
test('test 2', async ({ page }) => { ... }); // gets its OWN context
test('test 3', async ({ page }) => { ... }); // gets its OWN context
```

Each test gets its own **BrowserContext** automatically in Playwright — you don't have to set it up manually.

2. "Every test gets a fresh environment, even when multiple tests run in a single browser"

One Browser Running:

```
├── Context A → Test 1 (fresh environment)
├── Context B → Test 2 (fresh environment)
└── Context C → Test 3 (fresh environment)
```

Layman terms: It's like 3 people using the **same computer** but each person has their own **separate incognito window** — they share the browser app but nothing else.

3. "Context won't share cookies/cache with other browser contexts"

```
typescript
test('test 1', async ({ page }) => {
  await page.goto('https://example.com');
  // logs in, stores cookies
  await page.locator('#login').click();
  //  cookie stored in Context A
```

```
});

test('test 2', async ({ page }) => {
  await page.goto('https://example.com');
  // ❌ Context B has NO idea about the login from test 1
  // starts completely fresh — not logged in
});
```

Layman terms: Like clearing your browser history/cookies between each test. Whatever happened in Test 1 **stays** in Test 1.

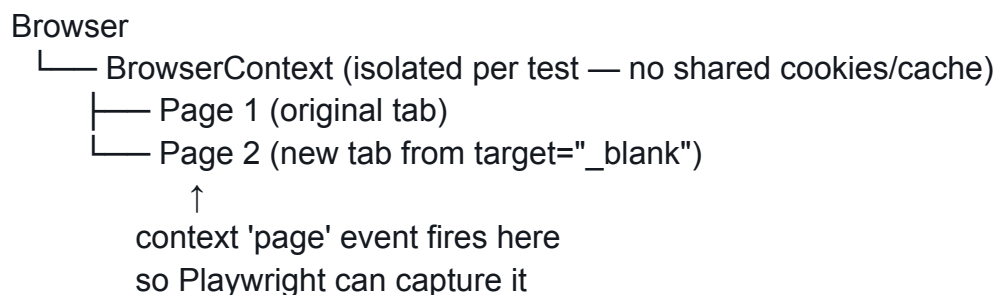
4. "The page event on BrowserContext can be used to get new pages created in the context"

Think of the context as a **manager** watching over all tabs. Whenever a new tab opens inside that context, the manager gets notified.

5. "This can be used to handle new pages opened by target='_blank' links"

typescript

The Big Picture



Summary Table

Statement	What it means simply
Isolated context per test	Each test gets its own incognito window
Fresh environment per test	No leftover data from previous tests

No shared
cookies/cache

Login in Test 1 doesn't affect
Test 2

page event on
context

Context notifies you when a
new tab opens

Handles
target="_blank"

You can catch and control
newly opened tabs

In VSCode:

Eg: `test("Handle New Page", async ({ context }) => {`

```
// 'context' fixture is used instead of 'page' fixture because we need to
// monitor the entire browser context for new tabs/pages being opened
```

```
const page = await context.newPage();
// Creates a brand new browser tab/page manually using the context
// This gives us full control over the page instance
```

```
page.goto("https://testpages.eviltester.com/styled/windows-test.html");
// Navigates the newly created page to the Windows Links Test URL
```

```
await expect(page).toHaveTitle("Windows Links Test Page | Test Pages");
// Assertion: Verifies the page has loaded correctly by checking its title
// Test will FAIL here if the title doesn't match — acts as a checkpoint
```

```
const pagePromise = context.waitForEvent("page");
// Sets up a LISTENER on the context BEFORE clicking
// This is crucial — it waits for a new page/tab to be opened
// Must be set up BEFORE the click, otherwise the new tab might open
// before we start listening and we would miss it
```

```
await page.getByRole('link', { name: 'External site in new tab' }).click();
// Clicks the link "External site in new tab" which has target="_blank"
// This triggers a new tab to open in the browser
// The context listener we set up above will now capture this new tab
```

```
const newPage = await pagePromise;
```

```
// Resolves the promise — meaning we now WAIT here until the new tab
// actually opens and assign it to 'newPage' variable
// 'newPage' now represents the newly opened tab we can interact with

    await newPage.getByRole('link', { name: 'Open book icon Read The Blog'
}).click();
    // Switches interaction to the NEW tab (newPage)
    // Finds the link "Read The Blog" by its accessible role and name
    // Clicks it to navigate further within the new tab

});
```